

The Hex 8 Architecture

COMS10015

Dr Tom Deakin

University of Bristol

Week 13

Outline

1. Housekeeping
2. Hex 8
3. The Hex 8 ISA
4. A micro-architecture for Hex 8
5. Timing
6. Modules and ModuleSim

Housekeeping

Life cycle of an instruction

“From sand to C”

This unit is all about popping the lid off the processors in your computer. Builds up from logic (maths), silicon (micro-electronics), programmable circuits (stored-program architecture).

So far, you've covered:

- ▶ Logic gates and circuits to build arithmetic circuits.
- ▶ Finite state machines, register machines.
- ▶ Stored-program computer (von Neumann architecture)
- ▶ Fetch-decode-execute cycle.

Next we'll look at life cycle of an instruction: creation by a compiler and execution on a real computer processor you'll build yourself.

Key concepts

- ▶ How do we put circuits together to build a processor?
- ▶ How do we control a processor's execution based on values (instructions) stored in memory?
- ▶ How do we bridge the gap between hardware and software?

COMS10015: Part 2

- ▶ Micro-architecture and Instruction Set Architectures (ISAs).
- ▶ Details on the memory hierarchy, including caches and virtual memory.
- ▶ Hardware support for software, including Operating Systems and debuggers.
- ▶ Software abstractions above the ISA/micro-architecture, including compilers and assemblers.

You'll build your own computer (in simulation) and write and build programs to run on it.

Expectations

- ▶ Two in-person lectures per week.
- ▶ Weekly lab slot.
 - ▶ One exercise sheet per week.
 - ▶ Used to reinforce things from the lectures.
 - ▶ Use the timetabled lab classes to ask for help and feedback.
- ▶ Weekly drop-in.
- ▶ Check back at Unit Intro lecture and Handout.

Remember:

20 credits $\approx$ 200 hours $\rightarrow \approx$ 8.3 hours per week over 24 weeks, i.e., about 1 working days per week.

5–6 hours/week is timetabled, leaving 2–3+ hours each week for *self-study*, revision, formative assessments,

Assessment

Summative: Summer Exam (70%)

Format: Short Answer Questions.

Content: everything in the unit, approx $\frac{1}{3}$ on TB-1 material, $\frac{2}{3}$ on TB-2 material.

Formative: Exams #3 & #4

Format: Multiple Choice Questions, upload plain-text file.

Deadline: Week 18 and Week 24.

Hex 8

Instruction Set Architecture

Instruction Set Architecture forms the interface between software and hardware. Describes how software controls the hardware.

An ISA defines (as a minimum):

- ▶ Word size.
- ▶ State (e.g., registers, memory).
- ▶ List of instructions and how they interact with the state.
 - ▶ What they do and how they are represented.
- ▶ Execution model.

Can write programs in the ISA (assembly programs).

Micro-architecture

Definition:

A micro-architecture is an *implementation* of an Instruction Set Architecture (ISA).

- ▶ Low-level design of the processor.
- ▶ Made up of functional blocks connected together.
- ▶ Functional blocks made up of logic gates.

The 8-bit processor

In this course, you are all going to build an 8-bit processor.

- ▶ Start with the ISA.
 - ▶ List of instructions, register information, etc.
- ▶ You'll build the micro-architecture.
 - ▶ The *implementation* of the ISA.

We'll walk through the details of the micro-architecture design.

The Hex 8 Architecture

Architecture designed by David May (Professor Emeritus) to show how a computer works.

Main features:

- ▶ Short instructions which are easy to decode, but with enough features.
- ▶ Memory of 256 bytes.
- ▶ Processor registers are one byte.
- ▶ Each instruction is one byte.
- ▶ Small number of registers. Some registers have a specific purpose.

Remember: one byte = 8 bits.

Note

The Hex 8 ISA and micro-architecture tooling is designed to be pedagogically useful, but as such has key limitations and aspects which do not reflect real modern processor designs.

The Hex 8 ISA

The Hex 8 ISA

All instructions are 8 bit.

- ▶ 4 bits opcode: representing an operation.
- ▶ 4 bit operand: representing data.

4 bit codes give $2^4 = 16$ different instructions.

Note:

We will be using little-endian.

The Hex 8 Registers

The processor state is represented by four 8-bit registers.

`pc` The program counter.

`oreg` The operand.

`areg` Left-hand operand and result of arithmetic.

`breg` Right-hand operand.

Left- and right-hand refer to the two inputs of a binary operation: e.g. adding two numbers.

The execution cycle has 3-stages: Fetch, Increment Program Counter (PC), Execute.

Hex 8 Instructions

Four classes of instruction:

- ▶ Data access: 9 instructions.
- ▶ Branching, jumping and calling: 4 instructions.
- ▶ Expression evaluation: 2 instructions.
- ▶ Prefix (special): 1 instruction.

Will introduce these in stages, adding more complexity to our micro-architecture implementation.

Hex 8: Expression evaluation

ADD $areg \leftarrow areg + breg$: add registers and store in *areg*

SUB $areg \leftarrow areg - breg$: subtract registers and store in *areg*

When computing subtraction, the register values are treated as two's complement.

Hex 8: Data access

LDAC $areg \leftarrow oreg$: load constant into $areg$

LDBC $breg \leftarrow oreg$: load constant into $breg$

These load immediate values from the instruction operand into registers.

LDAP $areg \leftarrow pc + oreg$: load address in program

Note:

There are 6 other memory instruction that use memory, which we will introduce in a later lecture.

Hex 8: Branching, jumping and calling

Branch instructions include conditional and unconditional relative branches.

BR $pc \leftarrow pc + oreg$: branch relative unconditional

Will always update the program counter by increasing it by the value in *oreg*.

BRZ if $areg = 0$ then $pc \leftarrow pc + oreg$: branch relative zero

BRN if $areg < 0$ then $pc \leftarrow pc + oreg$: branch relative negative

Update the program counter depending on value of *areg*.

Hex 8: Branching, jumping and calling

BRB $pc \leftarrow breg$: branch absolute

Set the program counter to the value of *breg*.

Used for function calls, which we will explain more in a future lecture.

Hex 8: Summary

Seen nine of the sixteen instructions:

ADD $areg \leftarrow areg + breg$: add registers and store in *areg*

SUB $areg \leftarrow areg - breg$: subtract registers and store in *areg*

LDAC $areg \leftarrow oreg$: load constant into *areg*

LDBC $breg \leftarrow oreg$: load constant into *breg*

LDAP $areg \leftarrow pc + oreg$: load address in program

BR $pc \leftarrow pc + oreg$: branch relative unconditional

BRZ if $areg = 0$ then $pc \leftarrow pc + oreg$: branch relative zero

BRN if $areg < 0$ then $pc \leftarrow pc + oreg$: branch relative negative

BRB $pc \leftarrow breg$: branch absolute

A micro-architecture for Hex 8

Micro-architecture of Hex 8

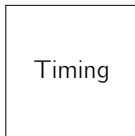
We can now start to design a *implementation* of the Hex 8 ISA.
Will construct a block diagram showing main parts of the processor, and how they connect together.

Hex 8 micro-architecture: Structure

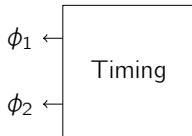


Data Path

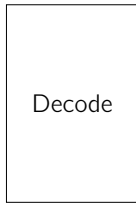
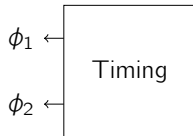
Hex 8 micro-architecture: Structure



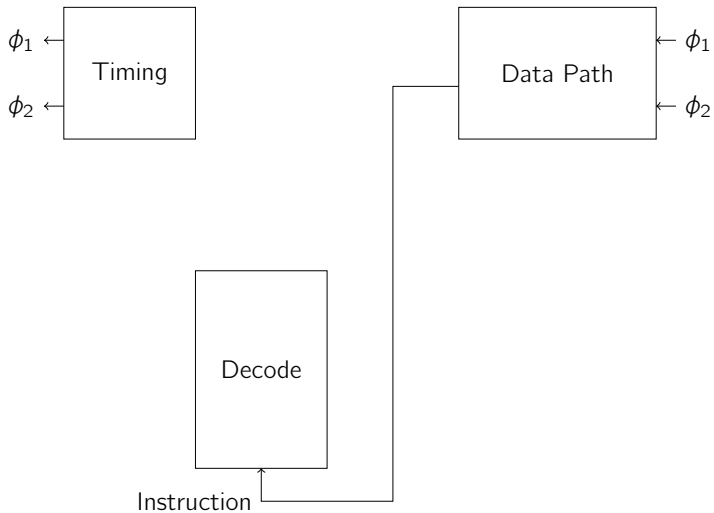
Hex 8 micro-architecture: Structure



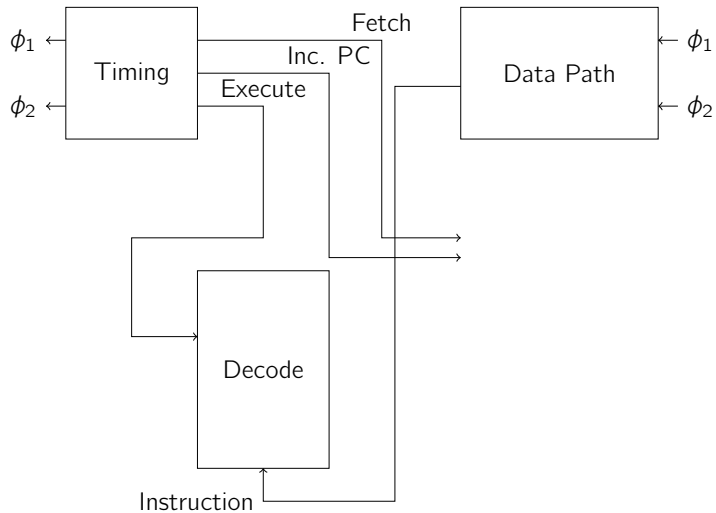
Hex 8 micro-architecture: Structure



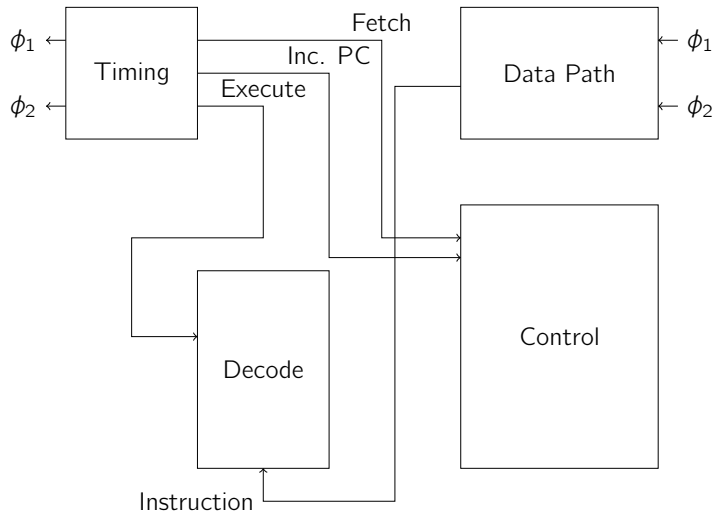
Hex 8 micro-architecture: Structure



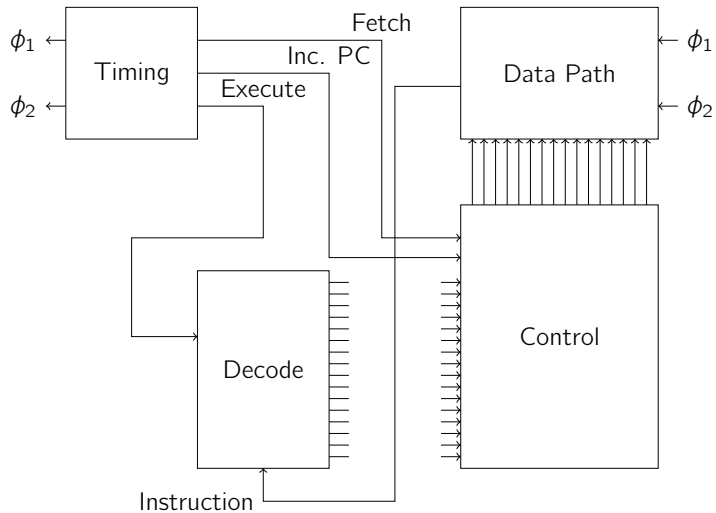
Hex 8 micro-architecture: Structure



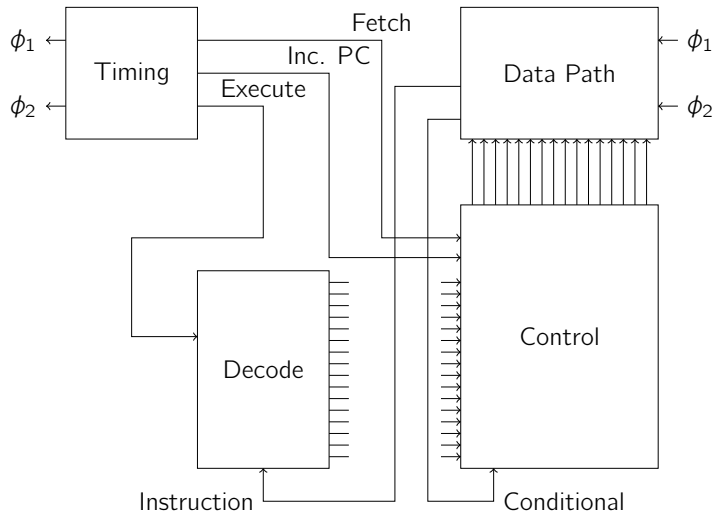
Hex 8 micro-architecture: Structure



Hex 8 micro-architecture: Structure



Hex 8 micro-architecture: Structure



Hex 8 micro-architecture

Timing 2-phase clock

Data Path Registers, arithmetic and memory

Decode Instruction decode, sending signal to Control

Control Turns on/off parts of data path depending on instruction

During the labs, we'll look at and implement each of these parts.

Timing

Timing unit

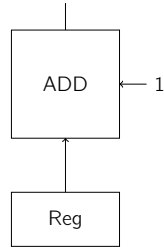
- ▶ Start with a register (will use 4-bit).



Reg

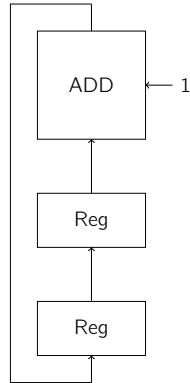
Timing unit

- ▶ Start with a register (will use 4-bit).
- ▶ Use an ALU to add 1 to the value.



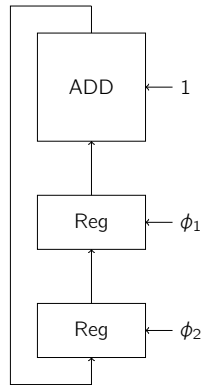
Timing unit

- ▶ Start with a register (will use 4-bit).
- ▶ Use an ALU to add 1 to the value.
- ▶ Want to feedback result back to add again — store in a second register.



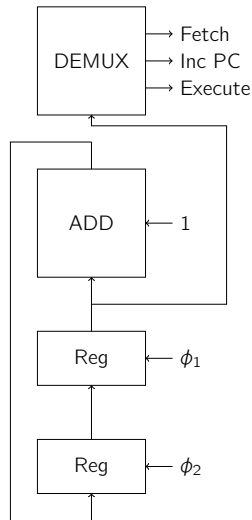
Timing unit

- ▶ Start with a register (will use 4-bit).
- ▶ Use an ALU to add 1 to the value.
- ▶ Want to feedback result back to add again — store in a second register.
- ▶ Two-phase clock so registers update in turn.

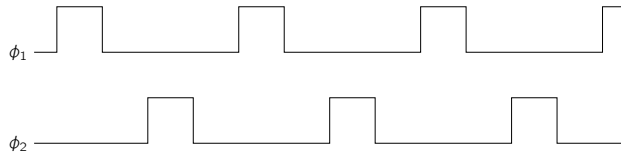


Timing unit

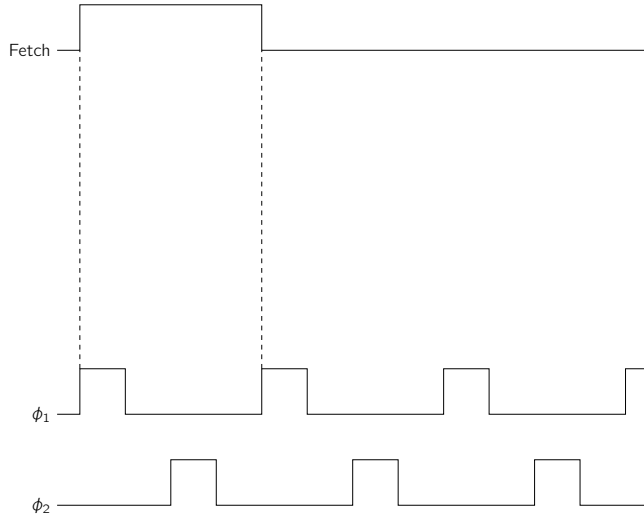
- ▶ Start with a register (will use 4-bit).
- ▶ Use an ALU to add 1 to the value.
- ▶ Want to feedback result back to add again — store in a second register.
- ▶ Two-phase clock so registers update in turn.
- ▶ Use DEMUX to split 4-bit number into signals.



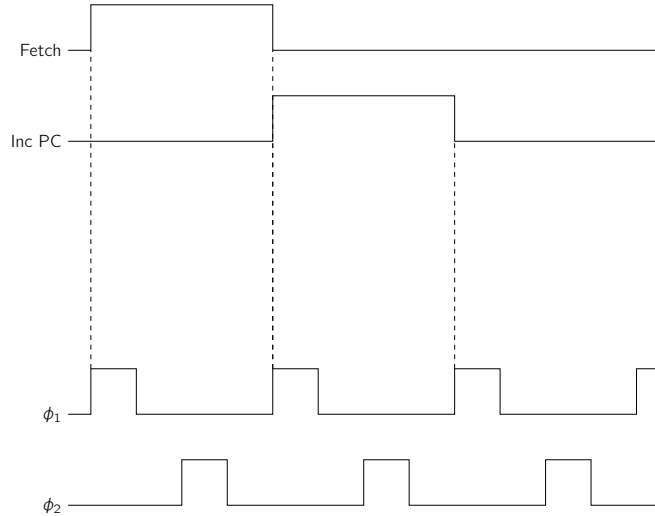
Timing unit: signals



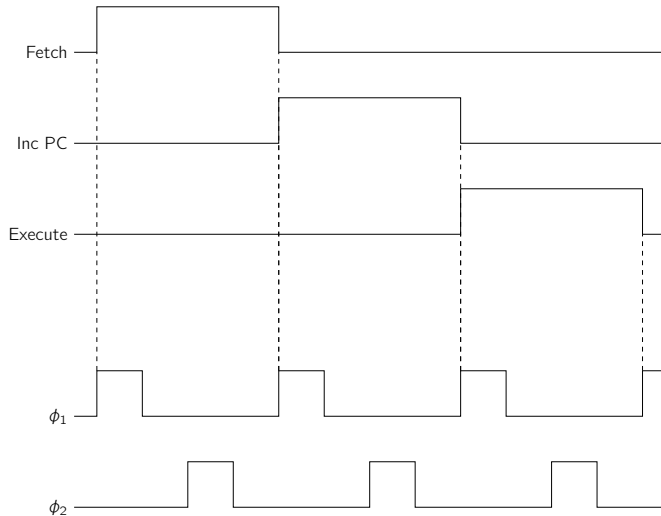
Timing unit: signals



Timing unit: signals



Timing unit: signals



Modules and ModuleSim

Modules

Bristol has developed some *Modules*:

- ▶ Physical functional blocks that can be put together to build a **computer**!
- ▶ Each Module is a 4-bit unit.
- ▶ Module for register, MUX, DEMUX, ALU, etc.
- ▶ Connect two 4-bit Modules to get 8-bit.
 - ▶ Control signals are propagated through.

Module details

Details of what each module does at

<https://teachingtechnologistbeth.github.io/ModuleSim/modules.html>.

Need to check the specific control signals required to set registers, select ALU operation, etc.

Lab classes

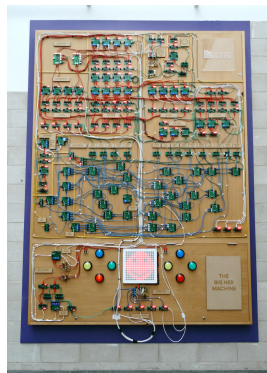
In the lab classes, you'll build the Hex 8 computer out of the modules (in simulation).

Big Hex Machine

Modules were once used to construct a giant 16-bit computer called *The Big Hex Machine*.

- ▶ Used to be on display in the Merchant Venturers Building.
- ▶ Design is very similar to the Hex 8 machine.

More details: <https://bighexmachine.github.io>



ModuleSim

- ▶ You'll use ModuleSim to build the Hex 8 computer instead of the physical modules.
- ▶ See lab sheet for details.